

COGNOMS:

[illegible]

NOM:

[illegible]

IMPORTANTE leer atentamente antes de empezar el examen: Escriba los apellidos y el nombre antes de empezar el examen. Escriba un solo carácter por recuadro, en mayúsculas y lo más claramente posible. Es importante que no haya tachones ni borrones y que cada carácter quede enmarcado dentro de su recuadro sin llegar a tocar los bordes. Use un único cuadro en blanco para separar los apellidos y nombres compuestos si es el caso. No escriba fuera de los recuadros.

Problema 1. (2,5 puntos)

Tenemos una aplicación de la que hemos programado dos versiones, una **secuencial (VS)** y una **paralela (VP)**. Se ha ejecutado **VS** en un sistema con una única CPU que funciona a una frecuencia de 2 GHz. Este programa ejecuta 50×10^9 instrucciones, realiza 20×10^9 operaciones de punto flotante y tarda 80×10^9 ciclos.

a) **Calcula** los MIPS y MFLOPS de **VS** en dicho sistema.

$$\text{Texe} = \text{Ciclos} / F = 80 \times 10^9 \text{ ciclos} / 2\text{GHz} = 40 \text{ seg}$$

$\text{MIPS} = \text{Instrucciones} / \text{Texe} \cdot 10^6 = 50 \cdot 10^9 \text{ instrucciones} / 40 \cdot 10^6 = \mathbf{1250 \text{ MIPS}}$

$$\text{MFLOPS} = \text{OpsPF} / \text{Texe} * 10^6 = 20 * 10^9 \text{ instrucciones} / 40 * 10^6 = 500 \text{ MFLOPS}$$

Sabemos que la version **VP** tiene una parte de su código que es perfectamente paralelizable (speed-up = numero de CPUs).

b) **Calcula** el porcentaje (%) de ese código que deberíamos paralelizar para conseguir un speed-up de 2 en un multiprocesador con 5 CPUs.

Ley de amdahl:

$$2 = 1/(1-p + p/5) \rightarrow p = 0.625 \rightarrow \mathbf{62.5\%}$$

Este sistema multiprocesador esta formado por los siguientes componentes:

	Numero	MTTF (horas)	Consumo	Fase Secuencial	Fase Paralela
CPU's	5	1.000.000	50 W	1	5
Placa Base	1	1.000.000	40 W	1	1
Fuente de alimentación	1	250.000	0 W	1	1
Discos duros	4	200.000	5 W	4	0
DIMMs de memoria	20	1.000.000	2,5 W	4	20

c) **Calcula** el tiempo medio hasta fallo del sistema suponiendo que todos los componentes están activos.

$$MTTF = 1/(5/1e6 + 1/1e6 + 1/2.5e5 + 4/2e5 + 20/1e6) = 20.000 \text{ Horas}$$

Al ejecutar **VP** en el sistema multiprocesador el tiempo de ejecución se ha reducido a 10 segundos y observamos que la fase secuencial representa el 25% del tiempo de **VP** y la paralela el resto del tiempo. Para reducir el consumo se apagan los componentes que no se usan, la tabla anterior muestra también el consumo de cada componente y cuantos se usan en cada fase.

d) **Calcula** la energía consumida al ejecutar **VP**.

$$P_{seq} = 1 \cdot 50 + 1 \cdot 40 + 4 \cdot 5 + 4 \cdot 2,5 = 120 \text{ W}$$

$$P_{par} = 5 \cdot 50 + 1 \cdot 40 + 0 \cdot 5 + 20 \cdot 2.5 = 340 \text{ W}$$

$$E = (0,25 \cdot P_{seq} + 0,75 \cdot P_{par}) \cdot T = (0,25 \cdot 120 + 0,75 \cdot 340) \cdot 10 = 2850 \text{ Joules}$$

COGNOMS:

NOM:

Problema 2. (2.5 puntos)

Dado el siguiente código escrito en C, que compilamos para un sistema linux de 32 bits:

```
typedef struct
{
    char c[3];
    short int M[3][3];
    char c2;
    short int e[5];
} sa;

typedef struct
{
    char a;
    sa vx[100];
    char *pc;
} sb;
```

- a) **Dibuja** como quedarían almacenadas en memoria las estructuras **sa** y **sb**, indicando claramente los desplazamientos respecto al inicio, el tamaño de todos los campos y el tamaño de los structs.

----- c <- 0	----- a <- 0
c[1] c[0]	- a
----- <- 2	----- vx <- 2=1+1vacio
- c[2]	
----- M <- 4=3+1vacio	.. vx[0] ..
M[0][0]	vx[0]
----- <- 6	----- <- 36
...	...

M[2][2]	
----- c2 <- 22	----- <- 3368
- c2	
----- e <- 24=23+1vacio	.. vx[99] ..
e[0]	vx[99]
----- <- 26	----- <- 3402
...	- -
-----	----- pc <-3404
e[4]	
----- <- 34	pc
	----- <- 3408
Tamaño: 34 bytes (alineado a 2)	Tamaño: 3408 bytes (alineado a 4)

- b) **Escribe UNA ÚNICA INSTRUCCIÓN** que permita mover **k.vx[37].M[2][1]** al registro **%ax**, siendo **k** una variable de tipo **sb** cuya dirección está almacenada en el registro **%ecx**. Indica claramente la expresión aritmética utilizada para el cálculo de la dirección.

La expresión aritmética para calcular la dirección del operando es: $@k+2+(37*34)+4+(2*3+1)*2 = \%ecx+1278$

y la instrucción es: **movw 1278(%ecx), %ax**

COGNOMS:

[illegible]

NOM:

[illegible]

Problema 4. (2.5 puntos)

Tenemos un procesador de muy bajo rendimiento que puede usar memoria virtual. Sabemos que el CPI ideal (asumiendo que todos los accesos a memoria tardasen 1 ciclo) de un programa es de 7 ciclos. El programa tarda 1.6s en ejecutarse, ejecuta 1×10^9 instrucciones y realiza un total de 1.3×10^9 accesos a memoria. La memoria principal de dicho procesador tiene un tiempo de acceso real de 10 ciclos y suponemos que no hay fallos de página.

- a) **Calcula** cuantos accesos a datos y cuantos accesos a instrucciones realiza el programa. **Indica** también cual es el CPI real del programa si no tenemos en cuenta la memoria virtual (no hay accesos a la tabla de páginas).

1x10⁹ accesos a instrucciones

0.3x10⁹ accesos a datos

CPI = 7 ciclos + 9 ciclos de penalización de ins + $9 \cdot 0.3$ ciclos de penalización de datos = 18.7 c/i

- b) **Calcula** el CPI del mismo programa si usáramos memoria virtual mediante paginación sin TLB.

1 acceso a memoria implicaría 2 accesos, 1 a la tabla de páginas y otro acceso al dato/instrucción en si. 2 accesos son 20 ciclos así que la penalización sería de 19 ciclos por acceso

CPI = 7 ciclos + 19 ciclos de penalización de ins + 19*0.3 ciclos de penalización de datos = 31.7 c/i

- c) **Calcula** el CPI real del sistema con paginación si usáramos un TLB con un 99% de aciertos. El TLB se accede siempre y tiene 1 ciclo de latencia..

$$\text{CPI} = 7\text{ciclos} + 10\text{ ciclos (penalización acierto TLB ins)} * 0,99 + 20\text{ ciclos (penalización fallo TLB ins)} * 0,01 + 0,3 * (10\text{ ciclos (penalización acierto TLB datos)} * 0,99 + 20\text{ ciclos (penalización fallo TLB datos)} * 0,01) = 20,13 \text{ c/i}$$

- d) **Calcula** el tamaño de la tabla de páginas de un solo nivel de 1 programa si el sistema tuviera 16 bits de direcciones lógicas, páginas de 8 KB y 15 bits de direcciones físicas.

8 entradas * (2 bits PPN por entrada + 1 bit presencia + 1 bit modificación) = 32 bits

Finalmente, al procesador anterior le hemos incorporado un TLB que asumiremos que acierta siempre y una cache unificada 3-asociativa con líneas de 16 bytes. En dicho sistema ejecutamos la siguiente función en alto nivel compilada sin optimizaciones:

```
int sumarvector(int v[]) {
    int i, suma;
    for (i=0; i<50000;i++)
        suma += v[i];
    return suma;
}
```

- e) Sabiendo que al ejecutar la función tanto las instrucciones como las variables locales están almacenadas en la cache unificada, ¿**cuantos** fallos tendrá al ejecutarse la función anterior? **Razona** el resultado.

Como la cache es de 3 vías sabemos que variables locales, $v[i]$ y las instrucciones no se interferirán entre ellas. Solo tendremos un fallo de $v[i]$ por cada 4 accesos (también puede haber uno extra si $v[i]$ no está alineado) -> 12500 o 12501 fallos.